Raport z miniprojektu

Metody probabilistyczne w uczeniu maszynowym

Zuzanna Szewczyk

9 kwietnia 2025

Spis treści

1 Analiza danych **2**
1.1 Wykresy 2

2 Przetwarzanie danych **3**

3 Trenowanie modelu **3**
3.1 Hiperparametry 4

4 Wyniki **4**

5 Modyfikacje modelu i regularyzacja **5**
5.1 Najlepsze wyniki 6
5.2 Wizualizacja predykcji dla najlepszych modeli 7
5.3 Wpływ parametrów na stratę 7
5.4 Wpływ rozmiaru zbioru treningowego 8

6 Rozwiązanie analityczne – Ridge Regression **9**
6.1 Najlepszy wynik Ridge Regression 9

7 Konkurs **10**

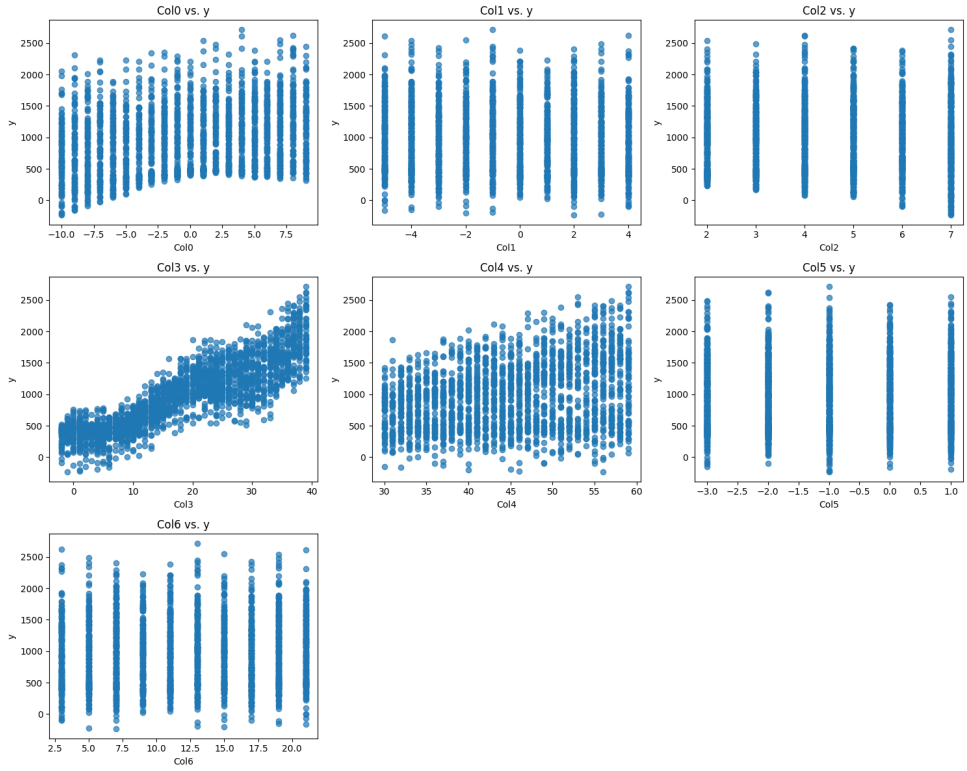
1 Analiza danych

Projekt rozpoczęłam od przeanalizowania danych. Prawie od razu napotkałam problem z danymi, ponieważ nie dostrzegłam, że liczby zmiennoprzecinkowe nie mają kropek tylko przecinki (zajęło mi to smutne 10 min). Następnie dodałam nazwy kolumn, bo nie wiedziałam czy później nie będzie mi łatwiej się do nich odnosić. Podzieliłam dane na wejściowe i wyjściowe i przemieszałam.

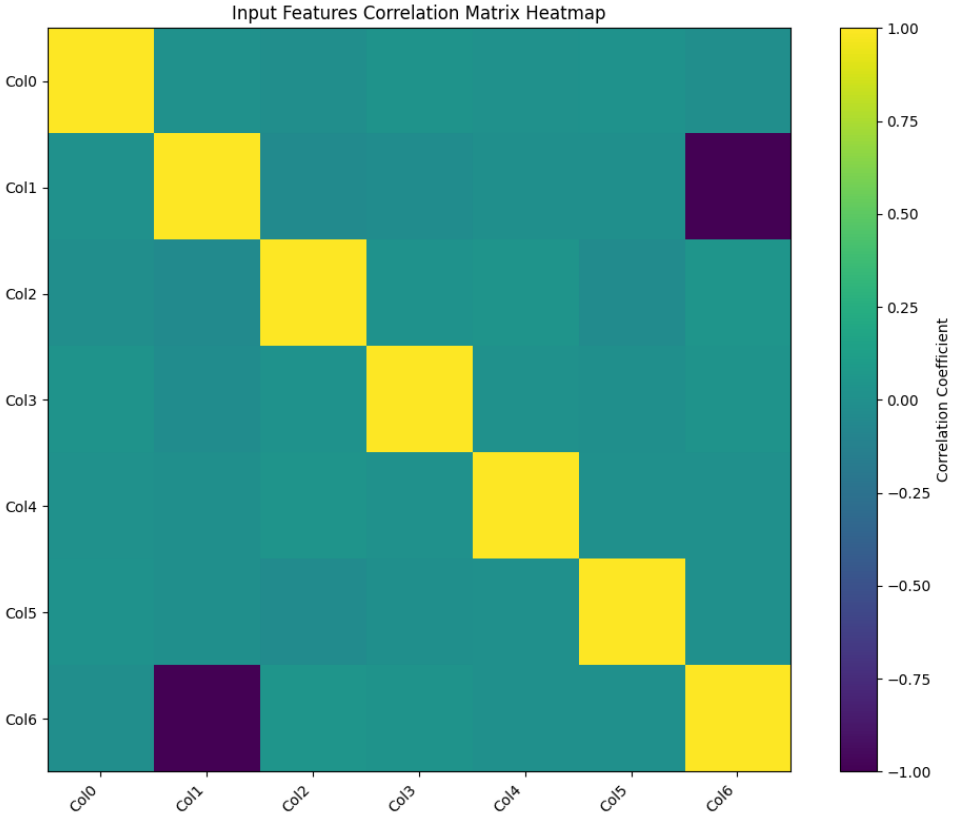
1.1 Wykresy

Dokonałam analizy danych na dwa sposoby:

- Wykresy rozrzutu kolejnych cech x w zależności od wartości wynikowej y (rysunek 1).
- Heatmapa korelacji cech z innymi cechami (rysunek 2). Tutaj się dość zdziwiłam, gdy zauważyłam, że większość z nich jest skorelowana ze współczynnikiem bliskim 0. Jedynie wartości z kolumny 1 i 6 były skorelowane ujemnie na -1.



Rysunek 1: Wykresy rozrzutu



Rysunek 2: Heatmapa korelacji

2 Przetwarzanie danych

Następnie zeskalowałam dane za pomocą własnego MinMaxScalera, który robi:

Przekształcenie opisane wzorem:

$$X_{std} = \frac{X - X_{min}}{X_{max} - X_{min}}, \quad X_{scaled} = X_{std}(max - min) + min \tag{1}$$

MinMaxScaler nie zmniejsza wpływu wartości odstających, ale liniowo skaluje je do ustalonego przedziału.

Przetestowałam także własny StandardScaler, który standaryzuje dane poprzez usunięcie średniej i skalowanie do wariancji jednostkowej:

$$z = \frac{x - u}{s} \tag{2}$$

gdzie u to średnia, a s to odchylenie standardowe.

3 Trenowanie modelu

Początkowo, aby zobaczyć tylko czy mój stochastyczny gradient descent w ogóle działa, podzieliłam dane w proporcjach:

- Zbiór treningowy: 60%

- Zbiór walidacyjny: 20%
- Zbiór testowy: 20%

Na początku testowałam wersję bez żadnej regularyzacji na tym stałym podziale na różnych funkcjach bazowych:

- sigmoid
- wielomiany od 1 do 5 stopnia
- leaky ReLU
- ReLU

Okazało się, że dane prowadziły do overflow już w samych funkcjach bazowych (np. wielomiany). Stąd wprowadziłam ograniczenie wartości funkcji bazowych. Ponieważ dalej miałam problem z eksplodującym gradientem, jego również zaczęłam ograniczać.

3.1 Hiperparametry

Mój podstawowy kod zaczął dawać zadowalające efekty nawet z losowymi hiperparametrami wymyślonymi przeze mnie:

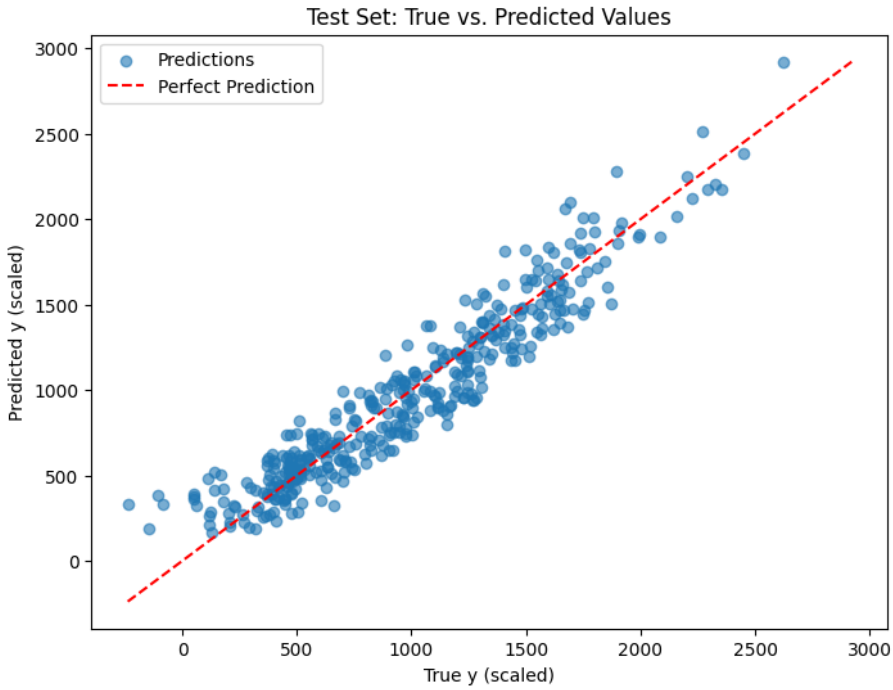
- `starting_bias` = 0.2137
- `learning_rate` = 0.01
- `epochs` = 31

4 Wyniki

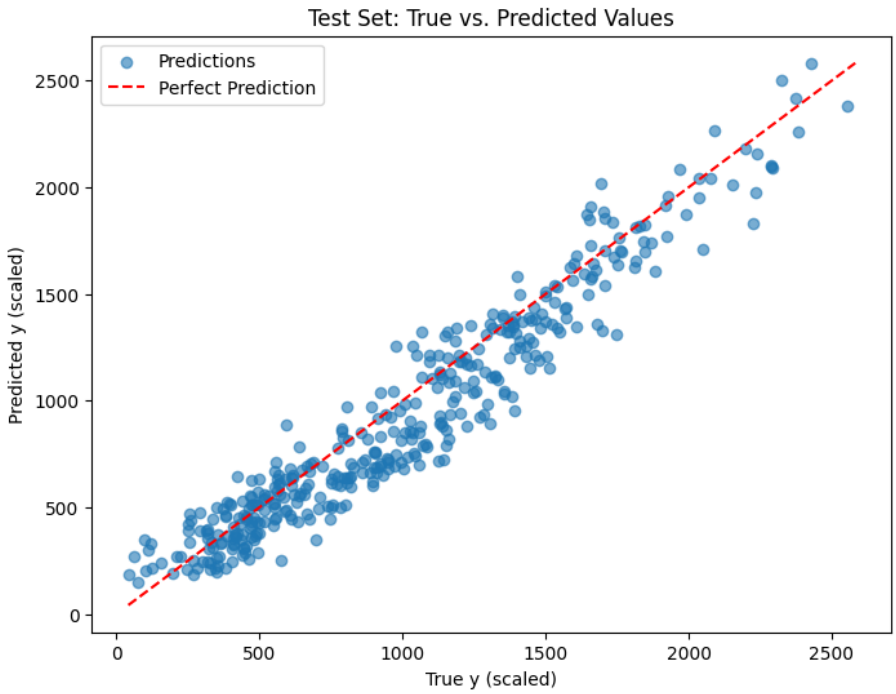
W tabeli 1 znajdują się wyniki dla kolejnych funkcji i odpowiadająca im strata MSE w zależności od użytego scalera. W przypadku MinMax Scalera, nie otrzymywałam warości ujemnych po skalowaniu, stąd ReLu, Leaky ReLu są tak naprawdę wielomianami 1 stopnia.

Funkcja bazowa	**MinMaxScaler**	**StandardScaler**
Sigmoid	1237859	1284969
ReLU	-	1126475
Leaky ReLU	-	1048897
Wielomian 1 stopnia	829014 -	1233838
Wielomian 2 stopnia	1004785	1009426
Wielomian 3 stopnia	292002	367364
Wielomian 4 stopnia	**26704**	**27402**
Wielomian 5 stopnia	54239	55496

Tabela 1: Tabela wyników MSE dla różnych funkcji bazowych oraz scalerów.



Rysunek 3: Predykcja modelu z wielomianem 4 stopnia i Standard Scalerem



Rysunek 4: Predykcja modelu z wielomianem 4 stopnia i MinMax Scalerem

5 Modyfikacje modelu i regularyzacja

Gdy mój podstawowy algorytm stochastycznego gradientu działał poprawnie, postanowiłam go zmodyfikować i dodać różne techniki regularyzacji:

- Regularyzacja **L1** (lasso),
- Regularyzacja **L2** (ridge),
- **Elastic Net**, czyli połączenie L1 i L2.

Aby znaleźć najlepsze parametry do tych metod, zaimplementowałam algorytm **Random Search**. Działa on na zasadzie losowego próbkowania przestrzeni hiperparametrów. W przeciwieństwie do Grid Search, nie sprawdza wszystkich możliwych kombinacji, tylko losowo wybiera zestawy parametrów i testuje ich skuteczność. Przy odpowiednio dużej liczbie prób często pozwala szybciej znaleźć dobre wyniki.

Zakresy, w których losowałam parametry:

- `learning rate`: $[10^{-4}, 10^{-1}]$
- `reg_lambda`: $[10^{-4}, 10^{1}]$
- `elastic_alpha`: $[0, 1]$

Każda kombinacja była trenowana przez 31 epok.

5.1 Najlepsze wyniki

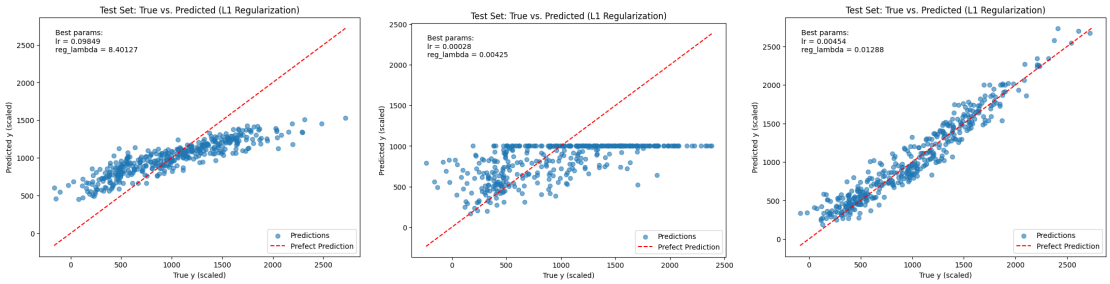
W tabeli 2 znajdują się najlepsze wyniki uzyskane po zastosowaniu Random Search, analogicznie do wcześniejszej tabeli (w tej wersji rozpatruje tylko MinMax Scaler, bo wczesnej rozpatrywałam oba aby zobaczyć, czy rReLU może dać jakąś dużą zmianę - ale nie dało :( ):

Regularyzacja / Funkcja bazowa	**Loss**	**lr**	**lambda**	**elastic alpha**
L1 - ReLU	120252	0.09849	8.40127	
L1 - Wielomian 3 stopnia	235860	0.00028	0.00425	
L1 - Wielomian 4 stopnia	21392	0.00454	0.01288	
L2 - ReLU	119982	0.09873	0.00289	
L2 - Wielomian 3 stopnia	240685	0.00028	0.04903	
L2 - Wielomian 4 stopnia	21225	0.00402	00031	
Elastic Net - ReLU	119232	0.09977	0.00051	0.62
Elastic Net - Wielomian 3 stopnia	250638	0.00028	0.00056	0.16
Elastic Net - Wielomian 4 stopnia	20087	0.00410	0.01093	0.79

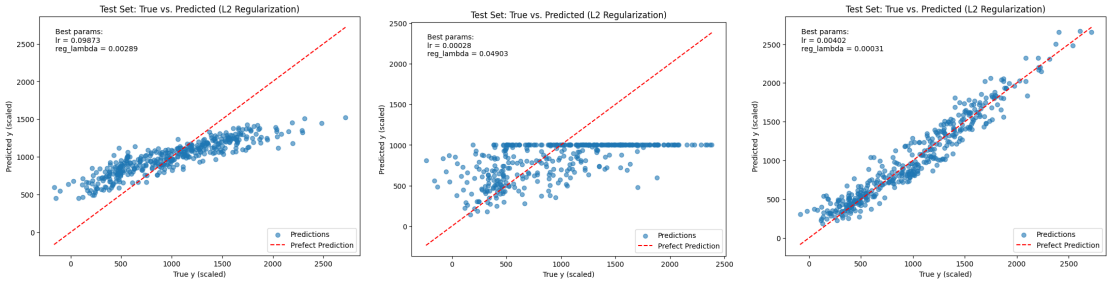
Tabela 2: Najlepsze wyniki MSE po zastosowaniu regularyzacji i Random Search.

5.2 Wizualizacja predykcji dla najlepszych modeli

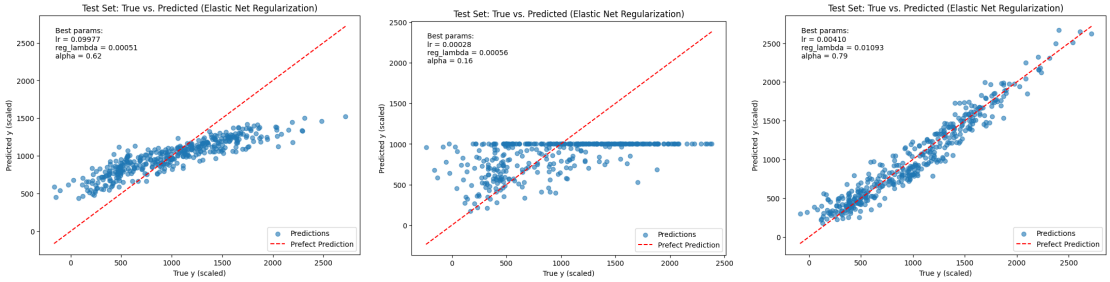
Regularyzacja L1: ReLU, wielomian 3. stopnia, wielomian 4. stopnia



Regularyzacja L2: ReLU, wielomian 3. stopnia, wielomian 4. stopnia



Regularyzacja Elastic Net: ReLU, wielomian 3. stopnia, wielomian 4. stopnia

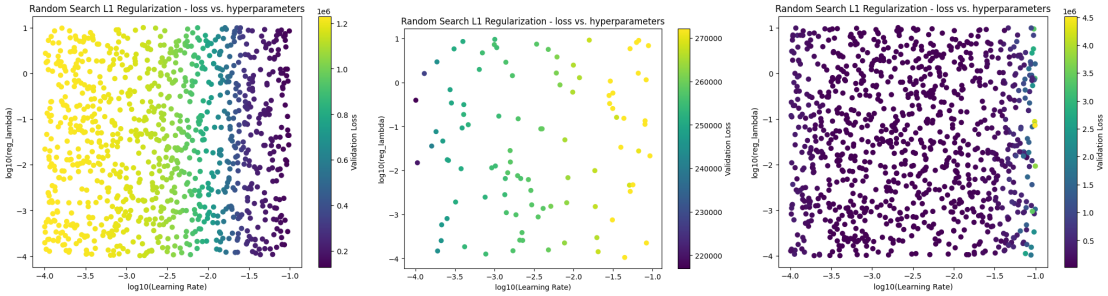


Rysunek 5: Porównanie predykcji modeli dla różnych regularyzacji (rzędy) i funkcji bazowych (kolumny).

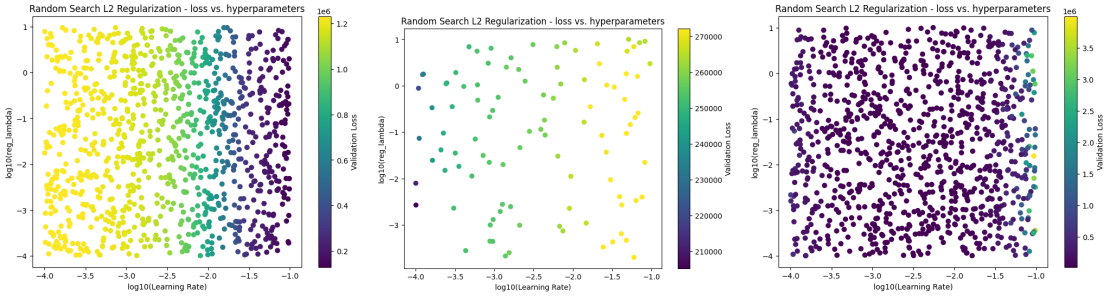
5.3 Wpływ parametrów na stratę

Aby lepiej zrozumieć, jak wartości hiperparametrów wpływają na wynikową stratę, stworzyłam wizualizacje:

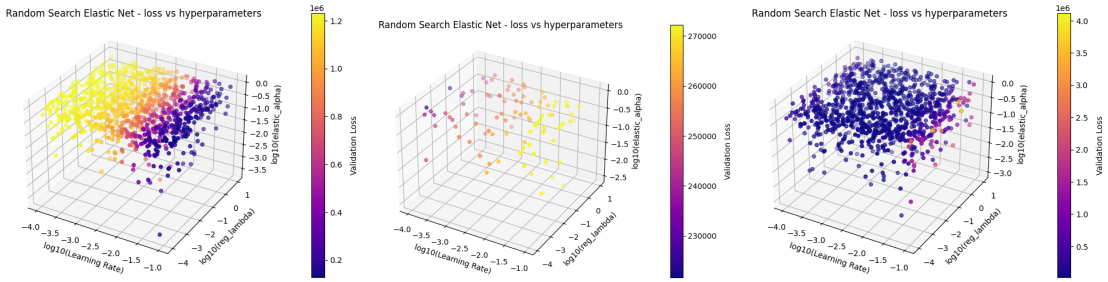
Wpływ log10(learning rate) i log10(reg_lambda) na stratę (L1)



Wpływ log10(learning rate) i log10(reg_lambda) na stratę (L2)



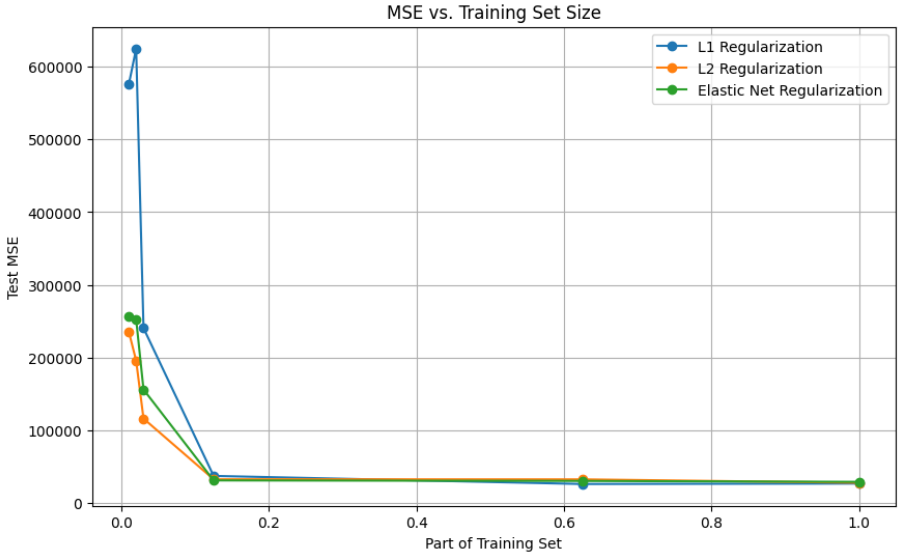
Wpływ log10(learning rate), log10(reg_lambda) oraz log10(elastic_alpha) na stratę (Elastic Net)



Rysunek 6: Porównanie wpływu hiperparametrów dla różnych regularyzacji (rzędy) i funkcji bazowych (kolumny).

5.4 Wpływ rozmiaru zbioru treningowego

Na koniec sprawdziłam, jak zmiana rozmiaru zbioru treningowego wpływa na jakość predykcji (stratę). Poniżej znajduje się wykres pokazujący zależność między procentem danych treningowych a stratą modelu:



Rysunek 7: Wpływ rozmiaru zbioru treningowego na stratę l1 przy parametrach $lr = 0.00291$ i $lambda = 0.00055$

6 Rozwiązanie analityczne – Ridge Regression

Oprócz trenowania modeli przy użyciu algorytmu stochastycznego gradientu, przetestowałam również rozwiązanie analityczne dla regresji liniowej z regularyzacją L2.

Rozwiązanie analityczne minimalizuje funkcję straty:

$$J(w) = \|Xw - y\|^2 + \lambda\|w\|^2 \tag{3}$$

gdzie X to macierz cech, w – wektor wag, y – wektor wartości oczekiwanych, a λ to współczynnik regularyzacji.

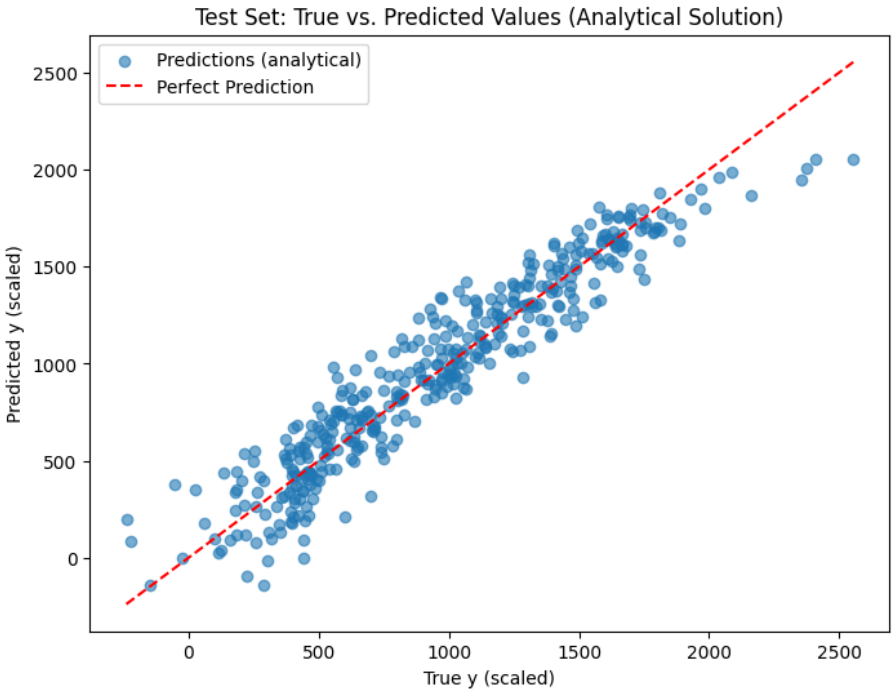
Optymalny wektor wag w można obliczyć ze wzoru:

$$w = (X^T X + \lambda I)^{-1} X^T y \tag{4}$$

W praktyce, zamiast bezpośredniego obliczania odwrotności $(X^T X + \lambda I)^{-1}$, używałam **pseudoinwersji** (`np.linalg.pinv` w Pythonie). Przy standardowej inwersji pojawiały się problemy numeryczne. Pseudoinwersja jest bardziej stabilna i pozwala na bezpieczne rozwiązanie układu.

6.1 Najlepszy wynik Ridge Regression

- `MSE:` **25720**



Rysunek 8: Predykcja modelu Ridge Regression dla najlepszej wartości λ

7 Konkurs

Wyniki eksperymentu

Dla wybranych parametrów: `learning rate (lr) = 0,00410`, `lambda = 0,01093`, `alpha = 0,79` oraz funkcji bazowej będącej wielomianem stopnia 4, uzyskałam:

Epoka	Loss
0	444 179,70
10	26 860,80
20	27 335,35
30	27 881,40
40	26 303,50
50	27 089,20
60	26 345,71
70	26 321,70
80	26 375,35
90	**26 248,67**

Końcowe parametry wytrenowanego modelu (Elastic Net):

- Wektor wag:

$$w_{\text{trained}} = \begin{bmatrix} 0{,}5906 & 1{,}6699 & -0{,}0251 & 2{,}3143 & 0{,}8075 & -0{,}0010 & 1{,}7034 \end{bmatrix}$$

- Bias: $b_{\text{trained}} = 1{,}8992$

Końcowa wartość RMSE dla całego zbioru danych wynosi: **153,70**.